Documentation as Infrastructure: Deterministic, Expert-Enriched Data Cards for AI-Usable, Verifiable, Production HPC System Data

I. Introduction

Responsible stewardship of research data imposes several concurrent requirements. The data must be available. It must be documented well enough that a reader can interpret it. There must be a binary criterion for judging whether that documentation is complete. Finally, the published artifact must be accurately usable by both human readers and machine agents, with a quantitative measure of how well each requirement is met in practice. This report describes a pipeline designed to satisfy those requirements.

The software comprises two cooperating subsystems that convert raw high-performance computing (HPC) job-accounting exports into standardized, human-reviewed dataset documentation in the Croissant JSON-LD format. The first subsystem, parbake, does the deterministic part of the work: it reads comma-separated value (CSV) files of arbitrary size in bounded memory, computes per-column descriptive statistics and AI readiness metrics, and generates a machine-readable metadata stub, termed a par-baked Croissant, that captures as much about the dataset as can be procedurally inferred. The second subsystem, bakery, ingests those stubs into a relational database, consults a corpus of domain-expert knowledge to supply the semantics that cannot be derived mechanically (field meanings, data types, units, decoding tables, licensing, responsible-AI attributes, and so on), and writes finished Croissant files that validate against the mlcroissant standard.

Figure 1 shows the outcome: a set of data cards, two formats for humans (text and Markdown) and one for machines (MLCommons's Croissant), describing the data contents, the quality the user can expect, how AI-ready the dataset is, and any semantic and domain knowledge identified along the way.

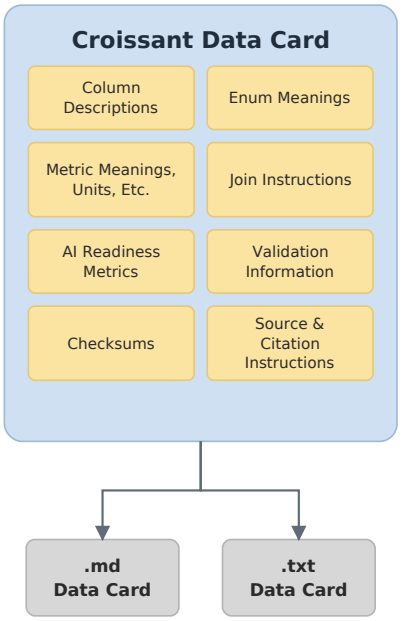


Fig. 1. Set of data cards produced for each dataset type, with one checksum for each instance of a dataset in the dataset type. The Croissant data card is a JSON-LD extended format pioneered by MLCommons. Each Croissant data card is versioned and considered canonical, and the Markdown and text files are generated from the Croissant file as conveniences for human readers.

II. Problems and Goals

Preparing ALCF data for consumption by users and AI systems poses challenges in three areas: the pipeline, the audience, and the documentation standard.

(Challenge 1): The pipeline must be deterministic yet performant enough to ingest multiple terabytes of data across different schemas and formats. ALCF systems produce operational data continuously, and the resulting corpus is large and heterogeneous. We need a standard that works for the whole facility, draws on existing techniques, and anticipates future uses of the data. Because the data will be released through the Office of Scientific and Technical Information (OSTI), the pipeline must also satisfy federal data standards. The pipeline must be parallelizable, fault-tolerant, and fully deterministic. It must also reuse stored knowledge across datasets with similar characteristics; the more of that reuse can be automated, the higher the pipeline's throughput.

(Challenge 2): The documentation must serve both human and machine readers. Good field-level context reduces errors and eases use, whether the reader is a person working alone, a program moving through a dataset, an AI assisting a human, or a fully automated system. Accurate documentation improves data use regardless of audience: for people working with unfamiliar data [1], [2], and for AI systems, where field-level documentation measurably improves how accurately a model works with a dataset [3].

(Challenge 3): The standard must be flexible enough to hold arbitrary expert knowledge, yet structured enough to be validatable and repeatable. Field meanings are essential, but the knowledge is fragmented: what a dataset contains,

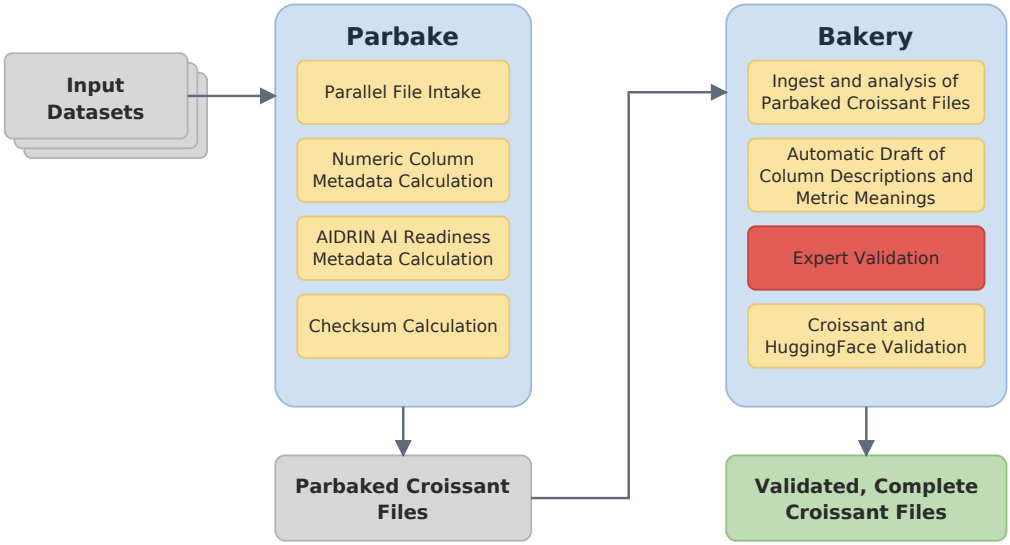


Fig. 2. Workflow for producing ALCF dataset data cards. Parbake reads through datasets in parallel, collecting deterministic values and AI-readiness metrics. The result is one "parbaked" (incomplete) Croissant file per dataset. These data cards are then ingested by Bakery. Bakery groups like datasets while preserving per-file measurements. Based on similarities to known dataset quantities, drafts are produced to further describe the datasets. Drafts and unknown quantities are presented to a user (a domain expert) for review and sign-off. The process concludes by running the finished Croissant data card through validators from HuggingFace and from MLCommons, as well as through workflow-specific checks. The output is a validated, versioned Croissant data card, one per dataset type, that has been signed by a domain expert. The human-readable .md and .txt data cards are produced automatically from the canonical Croissant file.

and what its fields mean, is known in detail by the people close to it, but that knowledge is scattered. Units, enum decodings, sentinel values, anonymization status, and known quirks are all needed to understand and use a dataset. The format must accommodate arbitrarily detailed information, and the standard should apply this knowledge to datasets and make best guesses where determinism fails, flagging assumptions for expert review. This keeps a human in the loop throughout, so the pipeline should minimize the effort that review requires.

To meet these challenges, we set the following goals for our documentation system:

- **Standardized** – extensible and tolerant of sparsity, since the data varies.
- **Familiar** – built from known, well-understood techniques and standards wherever possible, to reduce rework.
- **Versioned** – re-released with updated versioning to track changes transparently.
- **Detailed** – as much detail as is available, from field meanings to data-use caveats to known pitfalls.
- **Human-readable** – human-digestible formats at every step.
- **AI-digestible** – AI-readiness assessed with established techniques [4], and measured by whether the documentation improves an AI's accuracy on the data.
- **Forward-looking** – designed for compatibility with future strategic AI initiatives.

III. Data Card Pipeline

ALCF datasets will be released to the public through OSTI [5]. OSTI does not host data; it assigns a Digital Object Identifier (DOI), registers metadata with DataCite [6], and requires that the dataset already be posted at a publicly accessible location. The OSTI record points to a landing page that must resolve without a login and must either link to the data directly or explain how to request access [5]. OSTI compliance is a goal of this work, but the metadata OSTI and DataCite require is limited. We therefore design data cards as companions to every dataset.

OSTI and DataCite metadata describe a dataset (title, authors, description, license), but offer no place to record encodings, caveats, known qualities of the data, or field details such as meanings, timescale, and units. They also do not support robust versioning. To close this gap, we produce data cards per dataset: two for human readers in different environments, and one for machines. We reuse repeatable processes and existing technologies wherever possible, and design the pipeline to meet the goals above.

A. Existing Requirements and Techniques

Machine Card: Given the growing use of AI in supercomputer log analysis, we adopt Croissant, an MLCommons metadata format for machine-learning-ready datasets, for the machine-readable card [7]. Croissant is a JSON-LD (JSON for linked data) format that describes a dataset in four layers:

TABLE I
COMMANDS, OPTIONS, AND DEFAULT VALUES FOR RUNNING PARBAKE AND BAKERY.

command, option or key	*argument*	*does*
parbake		
parbakery.py	[DIRECTORY]	Measures every CSV inside DIRECTORY. Default: the current directory
--out	DIR	Output location. Default: ./parbake_output
--preview	[N]	Reads the first N rows of each file. This is the default, with N = 1,000
--full		Reads every line of each file.
--workers	N	Parallelism degree, one process each. Default: up to 8, capped by CPU count and file count
--separator	SEP	Field separator. Default: ,.
--batch-size	N	Rows read at a time per worker. Default: sized from each file's width to about 1,000,000 cells. Lower this if a run is killed for memory
--values-tracked	N	Distinct values counted per column before the count is stored as a minimum. Large values add to processing time. Default: 1,000
--values-shown	N	Values listed per column in the report. Default: 10
--no-identifier-check		Skips the check for non-anonymized identifiers
--no-checkpoints		Saves no progress
--checkpoint-every	N	Rows between saves. Default: 1,000,000.
--checkpoint-dir	DIR	Where progress is saved. Default: <out>/.parbake_checkpoints
--skip-existing		Skips any file whose outputs already exist in --out, and reuses its index entry
--batch-local-copies		Each worker copies its file to local disk, reads the copy, and deletes it before taking the next file. For shared filesystems such as Lustre
--local-copy-dir	DIR	Where --batch-local-copies stores copies. Default: $TMPDIR
bakery		
command line		
--import	DIR	Intakes a parbake output directory
--seed	PATH...	Reads finished Croissants, field dictionaries and documentation.
--bake --out	DIR	Write out validated Croissant files
--notes	PATH...	Lists every *note* in the Croissants under a path
--database	URL	Chooses the database; default ./bakery.sqlite, or BAKERY_DATABASE_URL

TABLE II
DOCUMENTATION OUTPUT BY SYSTEM SO FAR.

System	Dataset	Files	Size	Parbaked	Baked
Polaris	xalt_csv	3332	895 GB	✓	✗
	GPU Telemetry	801	329 GB	✓	✓
	hpcmlog	287G	287 GB	✓	✗
	log_syslog	199G	199 GB	✓	✗
	Syslog	1223	2.8 GB	✓	✓
	Job Metadata	62	377 MB	✓	✓
	Availability and Usage	296	1.2 MB	✓	✓
Aurora	Syslog	6977	8.1 TB	✓	✓
Metis	Pod Logs	64	3.8 GB	✓	✓

metadata, file descriptions, record sets for each field, and machine-learning semantics such as splits and labels [7]. Field types map to schema.org primitives. The format is flexible and extensible, and includes dedicated "Responsible AI" fields for data limitations, known statistical breakdowns, sensitive information, and intended uses. MLCommons publishes an open-source Python library and a validator [8]. We target Croissant 1.1, the current version of the specification [9].

Human Cards: Croissant files are verbose and deeply nested, so we also produce a human-facing description in .txt and Markdown [10]. Markdown is a lightweight plain-text markup: it is legible as raw text, renders cleanly on the web and in common tools, and works well with version control. Plain-text files render directly in terminal environments. Both are generated from the Croissant file.

B. Current Design

Conversion from Croissant to Markdown and plain text is deterministic, whereas the reverse conversion is not reliably reproducible, so we keep the Croissant data card as the canonical source. The pipeline is as follows:

1) Run parbake over a directory of files, producing "parbaked" data cards that hold everything the system can determine automatically from the source files. Assumptions are marked, and sentinel values keep a parbaked Croissant from validating before it is fully baked.
2) Import the parbaked files into the bakery database, applying any information the system can verifiably confirm from other files and prior answers, such as column descriptions from other files in the same dataset. Column descriptions and meanings are drafted automatically for later human verification.
3) Run bakery to enrich the cards with missing or relevant information. This pass is interactive: it presents the operator with the files that need information, drafts to edit and approve, or final validation.
4) Each completed Croissant is validated automatically, and matching Markdown and text cards are generated.

A human with domain expertise and access to the data must check every assumption this process makes before signing off on a card. Figure 2 shows the complete pipeline.

C. Downstream Use

Agentic workflows can be streamlined with *skills*: files given to an agent that supply prompts, instructions, and

TABLE III
METRICS COMPUTED BY AIDRIN AND COLLECTED BY PARBAKE'S --ENRICH PASS.

Group	Metric	What it tells us about the dataset
Completeness	Completeness	Missing-value rate per column
	Row-level completeness	Pct of rows where every required column is present
	Feature coverage ratio	Pct of columns whose non-null rate clears a threshold
	Null-count trend	Grouped missing values, indicates sudden quality drop
	Temporal completeness	Share of expected time intervals present, exposes gaps in a time range
References & units	File-reference validation	Whether paths stored in a manifest column resolve to real files
	Variable-unit validation	Whether column units agree with a unit-metadata sidecar
Distribution shape	Skewness	Asymmetry of each numeric column
	Kurtosis	Tail heaviness of each numeric column
	Outliers (IQR)	Proportion of values outside the inter-quartile range, per numeric column
	Constant feature count	Columns showing only a single value (candidate column to drop)
Uniqueness	Duplicates	Proportion of fully duplicated rows
	Duplicity by features	Duplicated rows from a subset of columns, e.g. by a join key
Relationships	Correlations	Pairwise association between columns (Pearson/Spearman, Theil's U)
	Max pairwise correlation	The single strongest correlation among numeric columns
	Feature relevance	Strength of each feature's relation to a chosen target column
Fairness	Class distribution	Imbalance for a categorical target as an Imbalance Degree score
	Representation rate	Relative representation across a sensitive attribute (e.g., username)
	Statistical rates	Class proportions within each sensitive group, if indicated
Privacy: anonymity	k-anonymity	Smallest group of records sharing the same quasi-identifier values
	l-diversity	Distinct sensitive values within each quasi-identifier group
	t-closeness	Gap between each group's sensitive distribution and the global one
	Entropy risk	Entropy-based re-identification risk across quasi-identifier columns
Privacy: risk & compliance	Single-attribute risk	Re-identification risk from each named column alone (unique-value counts)
	Multiple-attribute risk	Joint re-identification risk over column pairs via a Markov model
	HIPAA compliance	Pattern scan of chosen columns against 18 PHI categories
	Differential privacy	Noise (Laplace) required to meet a set ϵ

context for a task, including processes, limitations, guidelines, or code snippets relevant to it. We provide a skill (*skill_using_alcf_croissants.md*) for working with ALCF Croissant data cards. It helps the agent build Jupyter notebooks that explore a target dataset, and surfaces the caveats and usage information the agent should pass on to a user, such as missing values, privacy concerns, and citation instructions. The skill also tells the agent how to cite its use of a dataset correctly.

D. Documented Datasets

Table II lists the ALCF datasets processed to date and the pipeline stage each has reached. Additional datasets are being processed on an ongoing basis.

IV. AI ASSESSMENT

AI systems are now routinely used for data analysis, yet accurate interpretation of data remains a persistent weakness of these systems. This is especially true of the densely encoded operational data that ALCF produces, where a model with no grasp of domain-specific metric meanings, exit-code conventions, or sentinel values frequently produces fluent, confident, and *incorrect* answers.

Our objective is for AI systems to operate correctly on these data, both under human supervision and autonomously. We pursue this in two ways. First, we select a set of deterministic data descriptions that together compose the **AI-readiness** of a dataset. This shows whether the dataset is complete, balanced, and free of the structural problems that make downstream use unreliable, especially in machine learning. Second, we demonstrate the **AI-usability** of our data cards. This measures the accuracy and speed of a model answering questions about a dataset, ablated across different levels of documentation. This is a more complex and evolving target, but an essential one to characterize.

A. AI-Readiness

We assess the intrinsic readiness of each dataset with a series of standard measurements and statistical techniques,

TABLE IV

TEST QUESTIONS FOR THE TWO AI-USABILITY TESTS. THE FIRST SET OF QUESTIONS CONCERNED DATA FROM THE SUPERCOMPUTER AURORA, AND NO DOCUMENTATION ON THE DATASETS WAS PROVIDED. THE SECOND SET OF QUESTIONS CONCERNED THE SUPERCOMPUTER POLARIS, AND AN ABLATION STUDY WAS PERFORMED, SUPPLYING THE MODELS WITH DIFFERENT QUANTITIES AND QUALITIES OF DOCUMENTATION TO TEST THEIR ACCURACY AND SPEED. FOR THE AURORA BASELINE TEST, CLAUDE OPUS 5 ANSWERED CORRECTLY ALL BUT A13 AND A14. A FULL LIST OF QUESTIONS AND ANSWERS CAN BE FOUND IN THE APPENDIX TO THIS DOCUMENT.

#	Question
Aurora, Question Subset A	
A1	What is the total number of jobs with non-zero exit codes?
A2	What is the average runtime of all the jobs?
A3	What is the average runtime of all the non-zero exit code jobs?
A4	What is the average wall time of all the jobs?
A5	What is the average wall time of all the non-zero exit code jobs?
A6	What is the average queued wait time of all the jobs?
A7	What is the average queued wait time of all the non-zero exit code jobs?
A8	What is the average nodes used by all the jobs?
A9	What is the total number of kernel messages? Use the identifier column.
A10	What is the total number of kernel messages per job? Use the identifier column.
A11	What is the total number of kernel messages for jobs with nonzero exit code?
A12	What is the average number of kernel messages per non-zero-exit job, including failed jobs that had zero kernel messages?
A13	What is the total number of jobs with priority ≤ 3? Use the "priority" column.
A14	What is the total number of nonzero exit code jobs with priority ≤ 3? Use the "priority" column.
A15	What is the total number of jobs with the message "show_signal_msg" in the text?
A16	What is the total number of jobs with nodes used greater than 15 that have kernel messages?
Polaris, Question Subset B	
P1	What does `GPU_aggr_dbit_ecc` tell us?
P2	Does this dataset have memory utilization records?
P3	I need to see if GPU measurements correlate to abrupt ends to jobs and to syslog contents. How does this join work?
P4	I'm loading this job-accounting table into a warehouse and want to drop dead weight. Which columns are completely useless for Polaris, and why are they in the schema at all?
P5	How many core hours did Polaris consume in August 2026? Compute it two ways: the DJC core-hours column vs `cores_used` × `runtime_seconds` / 3600, and tell me which number you trust.
P6	Jobs in the "debug" queue fail far more often than "backfill-small". Give the failure rates for both, the top 3 failure codes in debug's August 2026 failures, and the mechanism behind the most debug-specific code.
P7	Users complain some nodes keep corrupting their GPU memory. In the August GPU telemetry, which compute node shows the highest double-bit ECC error counter, how many distinct jobs landed on that node during August, and can the telemetry tell me WHEN those errors occurred?

implementation-agnostic by design. Our implementation uses AIDRIN (AI Data Readiness Inspector) [4], which is available as a Python package, a command-line tool, or a web application. AIDRIN scores a dataset across many dimensions, including completeness, outliers, duplicates, class imbalance, feature relevance and correlation, fairness, re-identification risk, and compliance with FAIR metadata principles. The metrics we use, and their importance, are detailed in Table III. These are properties of the data, measured without reference to any particular model.

The resulting metrics serve two uses. First, internal staff can use them to improve datasets before release, giving a concrete standard for data-quality expectations. Second, they can be published as part of the data cards, giving downstream users a quantitative indication of data quality.

B. AI-Usability

We cannot measure directly whether a model understands a dataset, so we instead measure an observable proxy: whether it answers correctly when the answer depends on semantic or domain understanding, and whether our documentation changes the odds of a correct answer. This mirrors the BIRD benchmark [3], which measured text-to-SQL accuracy with and without domain knowledge. The baked Croissant files and optional skill described earlier are meant to supply that knowledge.

A caveat. AI-usability is a moving target. Models and techniques evolve rapidly, so the specific results reported here will age quickly. The demonstration is nonetheless necessary: the value of this documentation depends on whether it measurably improves outcomes for both human and AI users. *Initial results indicate that our documentation improves the usability of the datasets for AI agents; we evaluate two models that differ in size and quantization.*

A baseline. Some of the largest datasets at ALCF concern the supercomputer Aurora. Aurora comprises more than 10,000 nodes and produces a wide range of telemetry, metadata, and system output, resulting in large data volumes. As a baseline, we assess how accurately an AI agent can work with this data without documentation.

We evaluate Claude Opus 5.0 on a dataset of 107,327 rows,

TABLE V

RESULTS FOR POLARIS DATASETS AND QUESTIONS P1 THROUGH P8 USING NEMOTRON. **CORRECT**: THE ANSWER WAS TECHNICALLY RIGHT (✓) OR WRONG (✗). **QUALITY** (CORRECT ANSWERS ONLY): ★☆☆ RIGHT BUT VAGUE OR INCOMPLETE, ★★☆ RIGHT AND CLEAR, ★★★ RIGHT, CLEAR, AND SPECIFIC. **SPEEDUP** IS DATA ONLY TIME DIVIDED BY THE ARM'S TIME (BELOW 1 IS SLOWER). **TIME** IS WALL-CLOCK SECONDS; **TURNS** IS COMMANDS RUN. A DASH UNDER QUALITY MEANS A WRONG ANSWER.

Q	Why this question	Context	Correct	Quality	Time (s)	Speedup	Turns
P1	*Requires metric name decoding as well as field content decoding*	Data Only	✓	★★☆	1161.8	1.00	7
		Parbaked Croissant	✓	★★☆	415.5	2.80	12
		Baked Croissant	✓	★★★	344.0	3.38	8
		Baked Croissant + Skill	✓	★★★	646.0	1.80	11
P2	*Requires column name understanding as well as catching bad data*	Data Only	✓	★★☆	358.5	1.00	11
		Parbaked Croissant	✓	★☆☆	290.0	1.24	5
		Baked Croissant	✓	★★★	476.4	0.75	8
		Baked Croissant + Skill	✓	★★★	257.8	1.39	4
P3	*Requires complex joins across many schema*	Data Only	✓	★☆☆	828.7	1.00	16
		Parbaked Croissant	✓	★☆☆	1078.6	0.77	7
		Baked Croissant	✓	★★☆	1134.8	0.73	8
		Baked Croissant + Skill	✓	★★★	743.6	1.11	7
P4	*Requires analysis of column contents (or complete metadata)*	Data Only	✗	–	297.1	1.00	5
		Parbaked Croissant	✗	–	412.8	0.72	6
		Baked Croissant	✓	★★★	461.6	0.64	9
		Baked Croissant + Skill	✓	★★★	244.1	1.22	6
P5	*Concerns numerically validatable quantities but with judgment explanations*	Data Only	✓	★☆☆	508.2	1.00	5
		Parbaked Croissant	✓	★★☆	191.0	2.66	6
		Baked Croissant	✓	★★☆	481.0	1.06	4
		Baked Croissant + Skill	✓	★★★	311.3	1.63	5
P6	*Requires joins, enum decodings, and judgement explanations; example of failure analysis*	Data Only	✓	★☆☆	587.6	1.00	5
		Parbaked Croissant	✓	★☆☆	266.6	2.20	11
		Baked Croissant	✓	★★☆	332.7	1.77	5
		Baked Croissant + Skill	✓	★★★	492.0	1.19	8
P7	*Requires a join, metric decoding, and judgement; example of automated Operations tasks*	Data Only	✓	★☆☆	1475.3	1.00	20
		Parbaked Croissant	✓	★☆☆	398.4	3.70	8
		Baked Croissant	✓	★★☆	561.0	2.63	7
		Baked Croissant + Skill	✓	★★★	648.0	2.28	6
P8		Data Only	✓	★★☆	47.4	1.00	5
		Parbaked Croissant	✓	★★☆	47.4	1.00	5
		Baked Croissant	✓	★★☆	47.4	1.00	5
		Baked Croissant + Skill	✓	★★★	47.4	1.00	5

	Data Only	**Parbaked Croissant**	**Baked Croissant**	**Baked Croissant + Skill**
Correct answers (of 7)	6	6	**7**	**7**
Mean quality	1.14	1.14	2.43	**3.00**
Total time (s)	5217.2	**3052.9**	3791.5	3342.8
Mean time per question (s)	745.3	**436.1**	541.6	477.5
Total tool commands	69	55	49	**47**

TABLE VI

AGGREGATE COST AND QUALITY FOR NEMOTRON ON QUESTIONS P1 THROUGH P7. MEAN QUALITY COUNTS A WRONG ANSWER AS 0. BOTH BAKED CROISSANT ARMS ANSWERED EVERY QUESTION CORRECTLY, AND BAKED CROISSANT + SKILL HAD THE HIGHEST QUALITY AND REQUIRED THE FEWEST TOOL COMMANDS; PARBAKED CROISSANT YIELDED THE FASTEST RESPONSE TIMES.

32 distinct PBS jobs, 258 job-node pairs, and 5,977 syslog message rows. Across 16 queries, the model answered 14 correctly, *without* any dataset documentation. The questions are shown in Table IV.

Claude Opus 5.0 is a current, highly capable model at the time of writing. Its 87.5% accuracy (14 of 16) indicates that the raw data is sufficiently well-structured to answer most questions without documentation.

Ablation Study. Polaris is a well-characterized 560-node system with four NVIDIA A100 GPUs per node. Mature GPU telemetry, syslog, and job metadata are public (see Table II). We process these four datasets through our pipeline, producing parbaked Croissant files (lacking human knowledge, field meanings, and the like) and completed "baked" files, one per dataset. To gauge the effect of our documentation on a model's accuracy and speed, we ablate by supplying increasingly complete documentation, along with instructions for using it:

- Dataset only
- Dataset plus parbaked Croissant data card

- Dataset plus full Croissant data card
- Dataset plus full Croissant data card plus our agentic *skill_using_ALCF_croissants.md*

We measure (1) whether the agent returned a correct answer, (2) how much time it spent, and (3) how many turns it took. The comparison tells us three things: the value of documentation in general, whether the cost of the human enrichment performed in bakery is justified, and whether our agentic skill affects model accuracy.

We test NVIDIA-Nemotron-3.5-Super-VL-120B, quantized to 4 bits, selected because it fits within the memory of a single NVIDIA DGX Spark, a common on-premises AI workstation at the time of writing. We record the correctness of each answer, the time the model took, and the number of turns. Per-question results are given in Table V, and aggregate results in Table VI.

We focus this analysis on the results from `Nemotron`. Correctness was largely unaffected by documentation, as expected: `Nemotron` is a capable mid-sized model, and the data itself is reasonably clean. Every arm that received documentation was consistently more correct, faster, higher in answer quality, or lower in turns than the data-only baseline.

Quality. Responses were more concise, more complete, and easier to parse when the model was given our documentation, most clearly with a baked Croissant, or a baked Croissant plus our *skill_using_ALCF_croissants.md*. With the dataset alone, this model produced no incorrect answers, but its responses frequently drifted from the question, adding extraneous detail and context.

Speed. Answers were faster whenever the model did not have to read the data to find them. For the computational questions (P4, P5), the parbaked card contained sufficient statistical information and was often the fastest. When semantic meaning was needed (P1), the baked Croissant was usually fastest. Supplying the skill often increased response time, most likely due to the overhead of processing the larger documentation context.

Turns. Turn counts (fewer is better) were consistently lower with the baked Croissant and with the skill. The documentation gave the model instructions for common tasks, so it iterated less, especially where joins were involved (P2, P6, P7).

Joins. Questions requiring joins (P2, P6, P7) were consistently higher in quality, faster, and lower in turns as documentation increased. The model no longer had to discover where each field was located or what it was called: join information is recorded in the Croissants, and the skill states how to use it.

V. Availability

Repositories are public and can be found at:

•

Version and DOI: *waiting for public ALCF repositories.* Requirements include: Python 3.11 or newer, pandas, numpy, psutil, sqlalchemy, textual; pandoc (optional); mlcroissant (optional and required only by two tests). The two repositories together contain 649 tests, executable with pytest. Software documentation is provided in the READMEs, `NOTES.md` files, and `docs/` in each repository.

VI. Forward-Looking: Toward Standards and Automated Insights

The work described so far treats documentation as the end goal. Well-constructed documentation can also serve as a foundation for broader capabilities.

First, ALCF does not currently have a standard for data provenance and versioning, and we are not aware of a published standard for facility-wide data curation at any national laboratory user facility. This work begins to close that gap, and it can be extended further.

Related work at other facilities suggests directions: OLCF's EPIC platform uses a hierarchical multi-agent system to perform operational data analytics on the Frontier system, answering complex queries over telemetry and logs [11]. Notably, EPIC found that fine-tuned smaller models could outperform large foundation models on descriptive analytics, which mirrors our own goal of making these datasets usable by cheaper models.

Facility-wide operational analytics of this kind depends on exactly the standardized, machine-digestible documentation this pipeline produces. A natural next step is to build a comprehensive platform for facility data exploration using agentic AI by expanding on the provided agentic skills, and to provide access to the fully baked Croissant files.

Such a system could answer simple but precise questions such as "What was the node utilization percentage for Polaris from October 2 to October 18, 2026?" without requiring a domain expert, and would provide an interactive environment for more complex queries and lines of research.

References

[1] K. L. Boyd, "Datasheets for datasets help ML engineers notice and understand ethical issues in training data," *Proceedings of the ACM on Human-Computer Interaction*, vol. 5, no. CSCW2, pp. 1–27, 2021.

[2] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.

[3] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, X. Zhou, C. Ma, G. Li, K. C.-C. Chang, F. Huang, R. Cheng, and Y. Li, "Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[4] K. Hiniduma, S. Byna, J. L. Bez, and R. Madduri, "AI data readiness inspector (AIDRIN) for quantitative assessment of data readiness for AI," in *Proceedings of the 36th International Conference on Scientific and Statistical Database Management (SSDBM)*. Association for Computing Machinery, 2024, also available as arXiv:2406.19256.

[5] U.S. Department of Energy, Office of Scientific and Technical Information, "Scientific data — submission basics," https://www.osti.gov/stip/submit/submission-basics/datasets, 2024, accessed 3 September 2026.

[6] ——, "DOI assignment flowchart," https://www.osti.gov/stip/submit/submission-basics/doi-assignment-flowchart, 2024, accessed 3 September 2026.

[7] M. Akhtar, O. Benjelloun, C. Conforti, L. Foschini, P. Gijsbers, J. Giner-Miguelez, S. Goswami, N. Jain, M. Karamousadakis, S. Krishna, M. Kuchnik, S. Lesage, Q. Lhoest, P. Marcenac, M. Maskey, P. Mattson, L. Oala, H. Oderinwale, P. Ruyssen, T. Santos, R. Shinde, E. Simperl, A. Suresh, G. Thomas, S. Tykhonov, J. Vanschoren, S. Varma, J. van der Velde, S. Vogler, C.-J. Wu, and L. Zhang, "Croissant: A metadata format for ml-ready datasets," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 82 133–82 148. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/9547b09b722f2948ff3ddb5d86002bc0-Paper-Datasets_and_Benchmarks_Track.pdf

[8] MLCommons, "mlcroissant: Python library for the MLCommons croissant format," https://pypi.org/project/mlcroissant/, 2025, version 1.1.0. Source: https://github.com/mlcommons/croissant. Accessed 3 September 2026.

[9] MLCommons Croissant Working Group, "Croissant format specification, version 1.1," https://docs.mlcommons.org/croissant/docs/croissant-spec-1.1.html, 2026, accessed 3 September 2026.

[10] J. Gruber, "Markdown," https://daringfireball.net/projects/markdown/, 2004, accessed 3 September 2026.

[11] A. M. Karimi, W. Shin, J. Hines, T. Ghosal, N. S. Sattar, and F. Wang, "EPIC: Generative AI platform for accelerating HPC operational data analytics," https://arxiv.org/abs/2509.16212, 2025.